Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 28 (2014) 332 – 339

Procedia Computer Science

Conference on Systems Engineering Research (CSER 2014)

Eds.: Azad M. Madni, University of Southern California; Barry Boehm, University of Southern California; Michael Sievers, Jet Propulsion Laboratory; Marilee Wheaton, The Aerospace Corporation
Redondo Beach, CA, March 21-22, 2014

n-Tiered Test Automation Architecture for Agile Software Systems

Patrick Day

Ventyx, an ABB Company, 400 Perimeter Center Place, Ste. 500, Atlanta, GA 30332, USA

Abstract

This paper introduces a multi-tiered test automation architecture to optimize test automation in an Agile software development environment while increasing both the test coverage and depth of each tier. Test Automation is the act of converting manual test cases into automated scripts that can be executed autonomously. In general, testing accounts for roughly 60% of the overall development budget and approximately 50% of that is attributed to regression testing. In recent years software organizations have begun migrating to Agile software development practices and automated testing in hopes of reducing the cost, lengthy regression cycles, and time to market. Traditionally, test automation is conducted on stable, non-changing applications. In an Agile environment where the code constantly changes, automated test cases become obsolete and must constantly be refactored in order to provide meaningful feedback about the system's quality. In most instances the cost of maintenance of automated test code completely overshadows the entire automation effort and negates any possible Return on Investment (RoI). An n-Tiered Test Automation Architecture seeks to retain the RoI by abstracting the automation project into separate distinct tiers; Presentation, Business, Data, and Services. These abstractions allow automated testing to continue providing feedback despite the constant revision of the system. A case study was conducted using this method and the observations showed that the automation architecture was resilient to change while increasing the test coverage, the depth of testing, and the overall quality of the application under test.

© 2014 The Authors. Published by Elsevier B.V. Open access under CC BY-NC-ND license.
Selection and peer-review under responsibility of the University of Southern California.

Keywords: Software Engineering; Testing; Test Automation; Architecture; Agile Systems

1877-0509 © 2014 The Authors. Published by Elsevier B.V. Open access under CC BY-NC-ND license.
Selection and peer-review under responsibility of the University of Southern California.
doi:10.1016/j.procs.2014.03.041

1. Introduction

In recent years, Software Engineering has been adapting to the evolutions occurring in the global software development process in order to alleviate high cost, high complexity, increased time to market, maintenance difficulties, and decreasing end user satisfaction. In using Agile principles an organization is able to add new functionality and release an updated version very quickly. This includes implementing planning, development, testing, and gathering feedback all within short iterative cycles or sprints. Enterprise software projects that are developed in such a fashion are known as Agile Software Systems. These types of systems have proven to be more effective and is of better quality than those developed in the classic waterfall methodology[5, 21].

A major characteristic of a mature Agile development team is the incorporation of test automation as part of the Agile process. Test Automation has traditionally been completed against systems that are stable, meaning that there are very few changes, and generally focuses around the User Interface (UI). The constant state of change makes automating tests extremely difficult and in some instances impractical. Tests that have been automated in a previous sprint may be obsolete in the current sprint due the changes within the source code. The result is that the cost of maintenance completely overshadows the entire automation effort and negates any possible Return on Investment. An n-Tiered Test Automation Architecture improves testing by providing abstractions by separating the project into distinct tiers or application layers. The Presentation tier focuses on the UI and user workflows; Business tier concentrates solely on the business logic; Data tier is concerned with data flow and integrity; Services tier is dedicated to testing the communication between external systems using web services. These abstractions allow automated testing to continue providing feedback within each tier despite the constant instability of the system. Each tier will have a different framework that is appropriate for the functionality of that particular tier, and usually integrates with a continuous build.

This paper will give an in depth description of a test automation architecture for Agile Software Systems, and is organized as follows. Section 2 will cover some background information on regression testing, test automation, and continuous integration. I will follow up with design considerations in section 3. Sections 4 & 5 will cover the n-Tiered Automation Architecture in more detail and discuss a case study of an Agile Software System respectively, and wrap up the conclusion in section 6.

2. Background

This chapter introduces three topics geared toward building a foundation and giving context to the remainder of the paper.

2.1. Regression testing

Regression testing is a crucial to the development phase of the software lifecycle, as the purpose is to eliminate errors prior to production and deployment of the product. Regression test suites are accumulated and constructed over the course of development cycle as the product stabilizes and matures. The cost associated with both managing and executing regression tests have become a major obstacle faced by many organizations today. As software complexity (and size) increases, so does the size of the regression suite and time needed to execute it. This turns testing from an asset to a deterrent to success. In most cases lengthy regression cycle(s) can extend the overall product delivery and can have dire consequences when dealing with new market penetration. Therefore it comes as no surprise that software manufacturers would seek out ways to shorten regression cycle time and maximize their revenue. One common approach is to select portions of the code for testing, particularly those evaluated as having the highest risk. Selective testing has a higher risk of undetected errors that may not be identified prior to production or deployment, where the cost of correction is far greater. The challenge lies in finding a balance in choosing the regression sets; execute too few and potentially allow a major defect to creep into production, or doing too many and incur additional costs. Recent trends show that software developers using Agile methods are turning to test automation as the answer to finding the right balance. Because automated software tests are conducted unattended, they can also be made more effective by running them more frequently.

2.2. Test automation

Test Automation typically increases the depth and speed of the quality assurance process, it is the core of Agile Testing and a key element of successful agile development. Test Automation is the use of software to control the execution of tests, the comparison of actual outcomes to predicted outcomes, the setting up of test preconditions, and other test control and test reporting functions[13]. Test suites and test cases are tailored to the size of the project. On small to moderate sized applications a framework would work sufficiently. However, on large enterprise level projects where scaling, performance, and interoperability are necessary an automation architecture must be utilized.

A framework is an Application Programming Interface (API) used to facilitate the process of automating test cases. Examples of frameworks include Java's JUnit [10], Google's Robot Test Framework[18], or SeleniumHQ Selenium Web Driver [19]. Architecture is the seamless integration of multiple frameworks into one cohesive system [14]. From a Systems Engineering perspective a framework is likened to individual independent system, and architecture is a System of Systems (SoS) where two or more independent systems are integrated to achieve some new capability.

2.3. Continuous integration

Continuous Integration (CI) is a practice where small changes are integrated frequently. Each code change or integration is built, packaged, and tested, while its main aim is to prevent integration problems. The rapid adoption of Agile methodology on such a large scale has increased the attractiveness of CI. Most Continuous Integration servers on the market today have some sort of Application Programming Interface (API) for extending functionality. This has led to the development of a significant amount of open source plugins, greatly increasing their flexibility and usefulness. A CI server encompasses the capability to build the code, perform multi-level testing (unit, functional and integration) and includes deployment. This provides a host of metrics at each stage to track progress.

3. Design considerations

When embarking on a new automation project there are some key consideration that must be taken into account in order for the effort to be successful. This sections calls out those key concepts and attempt to provide some guidance into building the appropriate architecture.

3.1. Managing test data

One of the biggest challenges in software testing is to decide what test data to use and how and when to create it. In test automation overcoming this challenge becomes even more crucial. The issue with test data is that it has to be maintained. Data sets often become "stale" or obsolete due to business process changes. Stale data increases the maintenance over the lifetime of the automation project. Without a test management strategy in place any automation endeavor is sure to be ineffective. The best approach to mitigate this issue is with a clear and decisive Separation of Concerns (SoC), an abstraction layer between the test data and implementation code. Understanding the data needed and required to initiate the System Under Test (SUT) with the desired behavior is key to organizing a strategy that will be flexible and robust. There are a few approaches commonly used varying in complexity and robustness for supplying the correct information at the right time to the SUT. These approaches are discussed in the next two sections.

3.1.1 Data files

Storing test data in XML or in a Schema such as .xsd file is more effective than spreadsheets or .csv files because they enable a thorough description and definition of the test data. For predictable and static data sets this technique is the most efficient as schema files can be read and loaded into memory providing performance gains. Also file sizes are typically fairly small therefore, transporting them across the network or sharing them between systems is ideal. Trying to persist or save data with data files, however, adds unnecessary complexity to the overall

architecture. Persisting data similar to the functionality of a database requires additional mechanisms for storing and retrieving information, which introduces additional points of failure and maintenance overhead to the system.

Data within a spreadsheet or csv file can be highly dynamic, and information can be easily organized into tables and persistence can be effortlessly attained. This is the main benefit of using a spreadsheet, and for this reason most architecture frameworks will use this approach to manage their test data. Most automation tools allow database objects to connection to a spreadsheet; obtaining a lightweight fully functioning database, without the overhead of a database footprint. Portability is usually not an issue; spreadsheets can be easily saved as .csv files and can be shared between systems and networks. Depending on how much data being stored file size can become a concern and portability becomes negated. If copious amounts of data is being utilized it is recommended that a database is employed to avoid data corruption.

3.1.2 Database

Database (DB) management is the recommended solution for extremely large amounts of test data. An excellent indication that a database should be employed is when data files approach the Gigabyte threshold. Databases have all of the strengths of the previous two solutions such as flexibility in describing data, scalability with large data sets, highly extensible, platform independence, and data persistence (this is obvious). The trade-off however, is most DB installations have a substantial footprint including the amount of resources needed to maintain functionality, and the increased overhead for maintenance. There are light weight alternatives available that can be run as an embedded instance, reducing the digital footprint, but one must weigh the inherent benefit against the incurred cost.

3.2. Managing test objects

If managing test data is one of the biggest challenges in test automation, then managing test objects is the most important one. Test objects are the same as any other object or class present in Object-Oriented Programming (OOP). Test objects come in many different types and have the potential to collaborate between one or more tiers. For example DAO/ADO objects are data related and primarily used to interact with a data store (Data), but can also be used within the Business tier. Each object has a state (properties) and associated behaviors (methods); these are what allow test objects to interact with the SUT, and how they are managed defines which framework is appropriate which help guides how the architecture derived.

3.2.1 Keyword driven framework

Keyword-driven testing and table-driven testing are interchangeable terms that refer to an application-independent automation framework. This framework requires the development of data tables and keywords, independent of the test automation tool used to execute them and the test script code that "drives" the application-under-test and the data. Keyword-driven tests look very similar to manual test cases. In a keyword-driven test, the functionality of the application-under-test is documented in a table as well as in step-by-step instructions for each test [12]. There are number of tools/frameworks in the market that are built around keyword driven testing, FitNesse[6] and Google's Robot[18] are among the most popular. Research around keyword-driven frameworks has increased recently. Model-Based Testing (MBT), a novel approach to software testing, using models to generate tests and then using Robot to drive those tests[17].

3.2.2 Data driven framework

Data-driven test frameworks use input values to read from data files and to then load output values into variables by automated scripts. In this framework, variables are used for both input values and output verification values. Navigation through the program, reading of the data files, and logging of test status and information are all coded in the test script. This is similar to table-driven testing in that the test case is contained in the data file and not in the script; the script is just a "driver," or delivery mechanism, for the data[12].

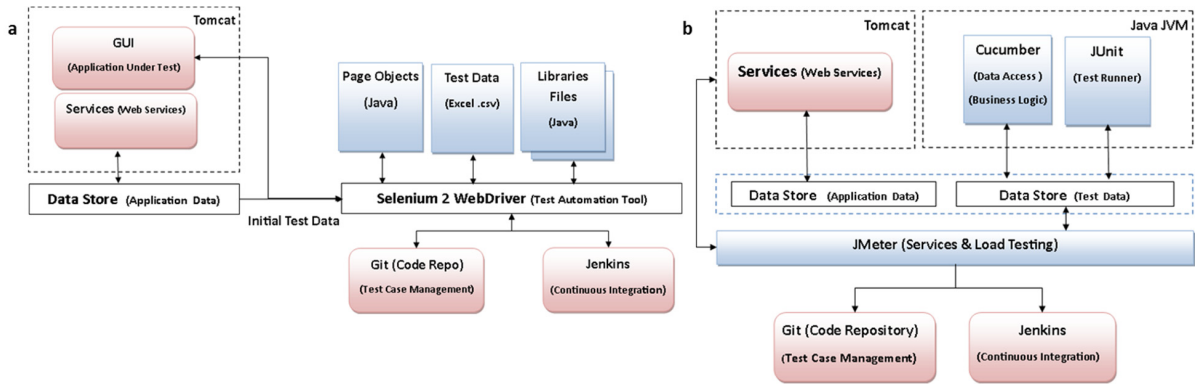


Fig.1. (a) Front-End Architecture; (b) Back-End Architecture

3.2.3 Hybrid framework

The most commonly implemented framework is a combination of all of the above techniques, pulling from their strengths and trying to mitigate their weaknesses. This hybrid test automation framework is what most frameworks evolve into over time and multiple projects. Figure 1 gives you an idea of how you could combine the approaches of the different frameworks[12]. Hybrid frameworks have proven to be highly effective at improving efficiency and productivity, while promoting code reuse, thus reducing maintenance[20].

4. An n-tiered test automation architecture

Software projects typically follow an n-tiered design process, where development is broken down into separate tiers or layers. A system typically exposes different interfaces to various types of clients. Each client may be interested in using or consuming that system in different ways. The user interface for example may interact with a service to generate the required information needed to display to the user. For instance a web service retrieving or persisting data to the database, or an external interface to other systems. Each interface can be grouped into a particular tier. The number of the layers greatly depends on the complexity of the project, indicating a larger number of interfaces to be tested.

Test automation requires significant software development, therefore, the effort should be treated the same as any other software project. Structuring automation into separate tiers creates abstraction which greatly increases the flexibility and depth of coverage, resulting in better overall quality of the system under test. There are no set number of tiers a system must have. Typically there are at least two: presentation and business. The following sections describe these and other popular tiers.

4.1. Presentation tier

The Graphical User Interface (GUI) is located within the presentation layer. Most automation projects focus too heavily on the GUI; it is usually the main entry point because it is the component that is presented to the user, so it's considered the most important. This approach is fundamentally flawed. In essence it is comparable to the proverbial saying of "putting all of your eggs in one basket", because the application's scope is so much more than just the UI. And by focusing automation resources solely on the user interface, the effectiveness of the project is limited by treating a substantial part of the system under test as a black box. Now depending on the complexity of the application, black box testing may be sufficient, however as complexity increases automation becomes unsustainable. Masquerading Business Logic as graphical objects and writing test scripts with this complex logic embedded into them becomes too difficult to develop and maintain. In fact the presentation tier should focus more on the user experience and less on the underlying logic. Once the Business Logic has been extricated, and handled in

its appropriate context, writing test scripts are greatly simplified. Not only will the efficiency and speed of scripting improve, but the amount of test coverage will also increase.

4.2. Business tier

The business tier contains the engine that produces the system requirements. Getting this tier correct is the end goal of the development team, so it naturally consumes a majority of development resources. Automation testing should reflect this importance. Understanding the business logic is crucial to successful automation testing. Exercising the various interfaces directly reduces the risk of gaps in testing. Testing through the GUI alone is not sufficient to completely test the many facets and interfaces that a system may present.

4.3. Data tier

The data access layer should not just be about persisting data. The focus should be expanded to include more than testing a connection and being able to pull and push data into a database, but test the quality and integrity of the data. From the beginning the Automation Architect should be able to participate in discussions pertaining to the initial schema design. In this phase the design documentation can be reviewed from a quality perspective. The goal in this tier is to ensure with a desired degree of confidence that the data that the application is using is correct. Most data stores provide a mechanism for data abstraction, and there are several frameworks available for creating data objects. Interfacing with a database using Data Access Objects (DAO) allows for a high degree of flexibility, robustness, and testability.

4.4. Web services tier

The web service layer of an application is focused on the interface of a given web service. The interface, either WSDL/WADL or REST "uniform contract", serves as a contract between the service provider and service consumer. The underlying implementation of the service is irrelevant; essentially operating as a black box, accepting input and generating an expected output. Testing web services usually take on two forms: functional & performance testing. In functional testing the goal is to validate that the service is indeed proving the expected results based on the agreed upon inputs. Under performance testing the intent is to verify that the service performs as expected under extraneous load or circumstances. There are a number of tools in market that specializes in web services testing that cover these two forms. The best results are usually gained when dynamic input data is interspersed with static data, which is essential to not only testing the functionality & performance, but for also uncovering emergent behaviors as well.

5. Case study

The following case study was conducted on an enterprise application that was built using Java technologies. The n-Tier Test Automation Architecture approach was applied to the second release of this product that was scheduled for 13 two week sprints. The project was designed to manage and maintain a system of record of user specific entities and perform domain specific analytics; while also coordinating the messaging and communication between those entities and external systems. As mentioned previously the technology stack was completely Java based; utilizing Tomcat application server[3], Java Messaging Service (JMS)[4], Java Persistence Architecture (JPA)[16], Web Services (JAX-WS)[11], and Google Web Toolkit (GWT)[8] which is a framework for User Interfaces. Maven 2[2] was used to manage dependencies across the entire project. In addition Artifactory, another open source tool, was used as a local artifact repository to decrease build times.

The automation architecture for this project had to be robust enough to handle the constant changes that would occur every sprint, and scale with the increased complexity of the application. To accomplish this all of the tiers were addressed resulting in two separate architectural views, a front-end and back-end.

5.1. Front-end

The Presentation tier encompasses the Graphical User Interface (GUI) depicted in figure 1a. This tier focuses on the testing of the basic GUI validation and user interactions using smoke style testing. The objective for the tests in this tier should be to validate that the correct information is displayed based on the user workflow. For this project the GUI is a web application built using Web 2.0 technology. Selenium Web Driver[19], an open source project, will be used as the primary automation tool. The application's data store was used to seed test data, which was used to drive testing. A source code repository called Git [7], which acts as a test case management system. This is done primarily because the automated test scripts are written in Java and checked in along with application source code. The Jenkins CI server [9] used to manage the execution of test as well visualizing the results.

5.2. Back-end

The back-end represents the Business, Data, & Web Services tier, which is illustrated by figure 1b. The purpose is to isolate and validate the business rules, data integrity, and web service functionality separately from the GUI. Validating algorithm design and system constraints are good examples of business logic that needs testing in these tiers. This project used Cucumber [15], which is a Behavior Driven Development (BDD) framework, in conjunction with JUnit, a unit testing framework [10], to handle the Business tier. Data Access Objects that were generated using JPA were used to interface with the data store, and used to assert the data against the business rules for accuracy. The web service testing was done with Apache JMeter[1], another open source tool. JMeter provided the ability to do functional and performance testing concurrently. Again all automated test scripts were stored in the source code repository, and Jenkins was used to integrate all tools together.

5.3. Observations

As the project progressed from sprint to sprint functionality was added and refined. Typically in Agile environments software requirements are not always fully fleshed out; therefore each new sprint can potentially introduced significant changes in the UI workflow, business rules, and even in the database schema. These types of changes usually wreaks havoc on an automation project due to the time needed for refactoring, while still trying to keep up with the testing. During this case study however, as change occurred maintenance became less of an issue. The abstraction between the tiers allowed for relatively quick refactoring. In some cases the automation code was turned around faster than the application source code. Another added benefit was that the test coverage improved dramatically. Each tier allowed focused testing within its context so it becomes easier to develop more elaborate scenarios, increasing the depth as well as coverage.

The previous release of the product had about 300 manual tests which took 120 man hours to execute and less than 100 automated tests with an execution time 75 minutes. As functionality was added, the inherent changes caused the automation effort to become unmanageable. Significantly more & more time was required, an average of 48 hours, to refactor and maintaining the tests, which stymied any attempt to add new automated tests. In contrast the second release using the n-Tier Automation Architecture, the product finished up with over 500 hundred manual test cases & over 600 automated functional, integration, and system tests that executed in less than 4 hours. The net saving exceed 720 man hours per sprint. In addition the automation discovered three "priority one" defects that manual testing alone wouldn't have found. This saved the organization $2.5M in support costs. The end result was that the product enjoyed the most successful release in that organization's history with the best quality of any of it other products.

6. Conclusions

This paper presented an n-Tiered Test Automation Architecture, a new approach for optimizing the development of automated tests in an Agile environment. The main principle behind the approach is abstracting the project out into separate tiers similar to that of enterprise level software development. The design considerations of an automation architecture was discussed, and the four commonly used tiers were described. A case study was

conducted where this architecture was applied to an Agile software system using Java technology during the second release of the product, and the observations were summarized in section 5.3. The study showed that by applying abstractions the test automation was able to cover a broader range of functionality while also allowing more detailed testing in critical areas. In addition, the cost of maintaining the test infrastructure was significantly reduced while increasing scalability and flexibility. The end result was that the product was released with higher overall quality compared to the initial release.

In closing as more software manufacturers embrace Agile principles, Test Automation becomes more prevalent and so does the issues that accompany the development of automated testing. Using this method will minimize a lot of these issues while increasing the overall RoI, and allowing Agile development teams to focus on what is really important; creating the best product possible.

References

1 'Apache Jmeter', <http://jmeter.apache.org/>.
2 'Apache Maven Project', <http://maven.apache.org/>.
3 'Apache Tomcat', <http://tomcat.apache.org/>.
4 David A. Chappell, Java Message ServiceO'Reilly Media, 2000. p. 240.
5 Vicente Lucena Eliane Collins ' Iterative Software Testing Process for Scrum and Waterfall Projects with Open Source Testing Tools Experience.', in 22nd IFIP International Conference on Testing Software and Systems, ed. by Adenilso Simão Alexandre Petrenko, José Carlos Maldonado 2010.p. 115 - 20.
6 'Fitnesse: The Fully Integrated Standaone Wiki and Acceptance Testing Framework', <http://fitnesse.org/>.
7 'Git -- Distributed-Even-If-Your-Workflow-Isnt', <http://git-scm.com/>.
8 'Gwt', <http://www.gwtproject.org/>.
9 'Jenkins: An Extendable Open Source Continuous Integration Server', <http://jenkins-ci.org/>.
10 'Junit: A Programmer-Oriented Testing Framework for Java', <http://junit.org/>.
11 Martin Kalin, Java Web Services: Up and Running. O'Reilly Media, 2009.
12 Michael Kelly, 'Choosing a Test Automation Framework'2003) <http://www.ibm.com/developerworks/rational/library/591.html>.
13 Dorothy Graham Mark Fewster, Software Test Automation, Effective Use of Test Execution ToolsAddison-Wesley, 2010).
14 Eberhardt Rechtin Mark W. Maier, The Art of Systems Architecting. 3rd ednCRC Press, 2009.
15 Aslak Hellesøy Matt Wynne, The Cucumber Book: Behaviour-Driven Development for Testers and Developer, 2012.
16 Merrick Schincariol Mike Keith, Pro Jpa 2: Mastering the Java Persistence ApiApress, 2009.
17 T.; Takala Pajunen, T.; Katara, M. Model-Based Testing with a General Purpose Keyword-Driven Test Automation Framework. in Software Testing, Verification and Validation Workshops (ICSTW), 2011.p. 242 - 51.
18 'Robot Framework: A Generic Test Automation Framework', https://code.google.com/p/robotframework/>.
19 'Seleniumhq: Browser Automation', <http://www.seleniumhq.org/>.
20 M. Xin Bai; Sadeghi, 'It Module Based Test Automation Framework', in Information Technology: New Generations (ITNG), 2011.p. 263 - 67.
21 Quan Yuan Zhi-gen Hu, Xi Zhang, 'Research on Agile Project Management with Scrum Method', in 2009 IITA International Conference on Services Science, 2009.p. 26-29.